

Chapter 3: Generating Random Variables

Lectures on Monte Carlo Theory — Leskelä & Vihola

Chapter Overview

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids
- 9 3.9 Bivariate Vectors: Conditional ITM and Copulas
- 10 3.10 Heaviness of Distributions
- 11 Summary

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.1 Inverse Transform Method — Motivation

Goal: Generate X with a given c.d.f. $F(x) = \mathbb{P}(X \leq x)$.

Key idea: If we can *invert* F , we can transform a $\mathcal{U}[0, 1)$ sample into a sample from F .

Generalized Inverse (Quantile Function)

$$F^{\leftarrow}(u) = \inf\{x : u \leq F(x)\}, \quad u \in (0, 1).$$

- $F^{\leftarrow}(u)$ is the *generalized inverse* (or quantile function) of F .
- If F is strictly increasing and continuous, $F^{\leftarrow} = F^{-1}$.
- $F^{\leftarrow}$ is non-decreasing and left-continuous.

Tail: Define $\bar{F}(x) = 1 - F(x)$. Then $\bar{F}^{\leftarrow}(u) = F^{\leftarrow}(1 - u)$.

Properties of the Generalized Inverse

Key Properties

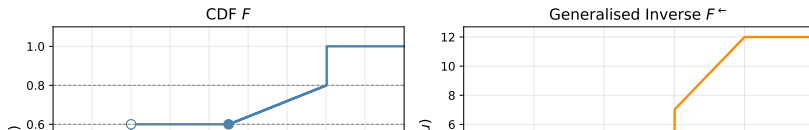
- Ⓟ1 $F^{\leftarrow}(u)$ is non-decreasing, left-continuous, with a right limit at u .
- Ⓟ2 If F is strictly increasing and $u \in F(\mathbb{R})$: $u = F(x)$ iff $F^{\leftarrow}(u) = x$.
- Ⓟ3 $F^{\leftarrow}(F(x)) \leq x$; if F is strictly increasing, $F^{\leftarrow}(F(x)) = x$.
- Ⓟ4 $F(F^{\leftarrow}(u)) \geq u$; equality holds when F is continuous.

Lemma 3.1.1 — Fundamental Equivalence

$$u \leq F(x) \iff F^{\leftarrow}(u) \leq x.$$

Consequently, if F is continuous, $F(F^{\leftarrow}(u)) = u$.

CDF and its Generalised Inverse



The ITM Theorem and Algorithm

Proposition 3.1.2 — The ITM Theorem

- a) If $U \sim \mathcal{U}[0, 1)$, then $F^{\leftarrow}(U) \sim F$.
- b) If F is continuous and $X \sim F$, then $F(X) \sim \mathcal{U}[0, 1)$.

Proof sketch:

$$\mathbb{P}(F^{\leftarrow}(U) \leq x) = \mathbb{P}(U \leq F(x)) = F(x).$$

Algorithm 6 — ITM

Require: c.d.f. F . **Ensure:** $X \sim F$.

- 1 Generate $U \sim \mathcal{U}[0, 1)$.
- 2 Set $X = F^{\leftarrow}(U)$.
- 3 **return** X .

Corollary 3.1.3: If F is continuous, the p -value $p = F(T)$ is uniform on $[0, 1)$.

3.1 Example: Exponential Distribution

Exponential Distribution $\text{Exp}(\lambda)$

$$F(x) = \begin{cases} 0 & x < 0 \\ 1 - e^{-\lambda x} & x \geq 0 \end{cases}$$

Mean = $1/\lambda$, Variance = $1/\lambda^2$.

Generalized inverse of $\bar{F}$:

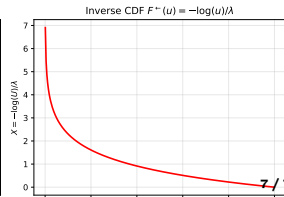
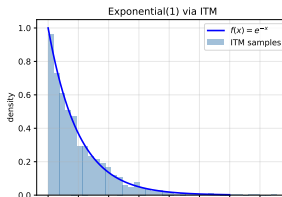
$$\bar{F}^{\leftarrow}(u) = -\frac{1}{\lambda} \log(u).$$

Algorithm 7 (ITM-Exp):

- 1 $U \sim \mathcal{U}[0, 1)$
- 2 $X = -\log(U)/\lambda$
- 3 return X

Listing 1: ITM for Exponential

```
1 import numpy as np
2 rng = np.random.default_rng(42)
3
4 def sample_exp(lam, n, rng):
5     """Generate n samples ~ Exp(lam) via ITM."""
6     U = rng.uniform(0, 1, n)
7     return -np.log(U) / lam
8
9 X = sample_exp(lam=2.0, n=5000, rng=rng)
10 print(f"Mean: {X.mean():.4f} (theory: {0.5:.4f})")
11 print(f"Var: {X.var():.4f} (theory: {0.25:.4f})")
```



3.1 Pareto Distribution (Heavy Tail)

Pareto Distribution $\text{Par}(\alpha)$

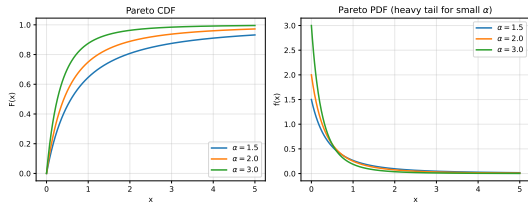
$$F(x) = \begin{cases} 0 & x < 0 \\ 1 - \frac{1}{(1+x)^\alpha} & x \geq 0 \end{cases}$$

- α — shape parameter (tail index)
- $\mathbb{E}X = \frac{1}{\alpha-1}$ (if $\alpha > 1$)
- Heavy-tailed: $\mathbb{E}e^{sX} = \infty$ for all $s > 0$

ITM: $\bar{F}^{\leftarrow}(u) = u^{-1/\alpha} - 1$

Algorithm 8 (ITM-Par):

- 1 $U \sim \mathcal{U}[0, 1)$
- 2 $X = U^{-1/\alpha} - 1$
- 3 **return** X



Pareto CDF (top) and PDF (bottom) for $\alpha = 1.5, 2, 3$.

Smaller $\alpha \Rightarrow$ heavier tail.

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.2 ITM Variants for Lattice Distributions I

Setting: X takes values in $\{0, 1, 2, \dots\}$ with probability function (p_k) , $k = 0, 1, \dots$

Proposition 3.2.1 — Discrete ITM

Let $U \sim \mathcal{U}[0, 1)$. The random variable

$$Y = \min \left\{ k : U \leq \sum_{i=0}^k p_i \right\}$$

has probability function (p_k) .

Algorithm 9 (ITM-d): sequentially add probabilities until cumulative sum $\geq U$. Expected number of loop iterations $= \mathbb{E}Y + 1$.

3.2 ITM Variants for Lattice Distributions II

Efficient variant (Algorithm 10, ITR): When the ratio $c_{k+1} = p_{k+1}/p_k$ is easily computable (e.g., Binomial, Poisson, Negative Binomial), use the recursion directly — avoids storing the full PMF.

The $(a, b, 0)$ class of distributions

A PMF satisfies $p_k = p_{k-1}(a + b/k)$ for $k = 1, 2, \dots$

- **Binomial** $B(n, p)$: $c_{k+1} = \frac{p(n-k)}{(1-p)(k+1)}$
- **Poisson** $Poi(\lambda)$: $c_{k+1} = \frac{\lambda}{k+1}$
- **Neg. Binomial** $NB(r, p)$: $c_{k+1} = \frac{(r+k)p}{k+1}$

3.2 ITM Variants for Lattice Distributions III

Geometric Distribution $\text{Geo}(p)$

PMF: $p_k = (1 - p)p^k$, $k = 0, 1, \dots$

Mean = p/q where $q = 1 - p$.

Closed-form ITM:

$$X = \lfloor \log U / \log p \rfloor \sim \text{Geo}(p).$$

Proof:

$$\mathbb{P}(X \geq k) = \mathbb{P}(\lfloor \log U / \log p \rfloor \geq k) = \mathbb{P}(\log U \leq k \log p) = \mathbb{P}(U \leq p^k) = p^k.$$

3.2 ITM Variants for Lattice Distributions IV

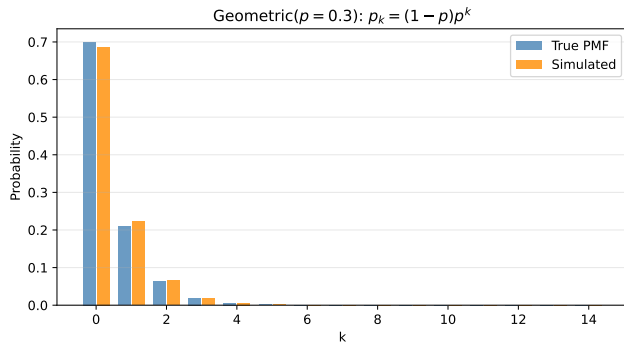


Figure: Geometric($p = 0.3$) via closed-form ITM.

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods**
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.3.1 Binomial $B(n, p)$ I

Representation as Sum of Bernoulli Trials

$$X \stackrel{\mathcal{D}}{=} \sum_{j=1}^n \xi_j,$$

where $\xi_1, \dots, \xi_n$ are i.i.d. with $\mathbb{P}(\xi_j = 1) = p$.

Algorithm 11 (DB):

- 1 $X \leftarrow 0$
- 2 **for** $i = 1$ **to** n :
 - Generate $U \sim \mathcal{U}[0, 1)$
 - **if** $U \leq p$: $X \leftarrow X + 1$
- 3 **return** X

This is $O(n)$. For large n with moderate np , the ITR algorithm (based on the recursion) is faster.

3.3.1 Binomial $B(n, p)$ II

3.3.2 Erlang Distribution $\text{Erl}(n, \lambda)$

$X = Y_1 + \dots + Y_n$ where $Y_i \stackrel{\text{i.i.d.}}{\sim} \text{Exp}(\lambda)$.

Since $Y_i = -\log(U_i)/\lambda$:

$$X = -\frac{1}{\lambda} \log(U_1 \dots U_n) = -\frac{\log(\prod_{i=1}^n U_i)}{\lambda}.$$

$$\text{PDF: } f(x) = \frac{1}{\Gamma(n)} \lambda^n x^{n-1} e^{-\lambda x}, \quad x \geq 0.$$

Also: $\text{Erl}(n, \lambda)$ is a special case of $\text{Gamma}(n, \lambda)$.

3.3.3 Poisson Distribution $\text{Poi}(\lambda)$

Renewal Process Characterisation

Let $\tau_1, \tau_2, \dots$ be i.i.d. $\text{Exp}(1)$. Count the number N of partial sums $\leq \lambda$:

$$N = \max\{i \geq 1 : \tau_1 + \dots + \tau_i \leq \lambda\}.$$

Proposition 3.3.1: $N \stackrel{\mathcal{D}}{=} \text{Poi}(\lambda)$.

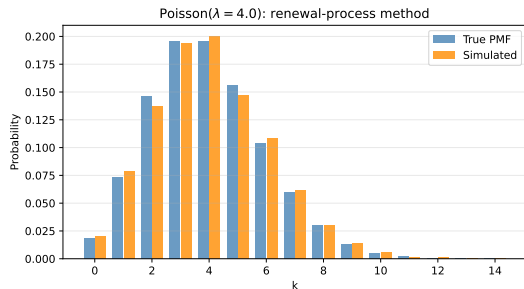
Simplified using $\tau_i = -\log U_i$:

$$N = \max\left\{n \geq 0 : \prod_{i=1}^n U_i \geq e^{-\lambda}\right\}.$$

(*convention:* empty product = 1.)

Algorithm 12 (DP):

- 1 $X \leftarrow 0, U \sim \mathcal{U}[0, 1), S \leftarrow -\log U$
- 2 **while** $S < \lambda$:
 - $U \sim \mathcal{U}[0, 1)$
 - $S \leftarrow S - \log U$
 - $X \leftarrow X + 1$
- 3 **return** X



3.3 Python Code: Poisson and Erlang

Listing 2: Ad hoc methods for Poisson and Erlang

```
1 import numpy as np
2 rng = np.random.default_rng(0)
3
4 def sample_poisson(lam, rng):
5     """Renewal-process algorithm for Poi(lam)."""
6     U = rng.uniform()
7     S = -np.log(U)
8     X = 0
9     while S < lam:
10         U = rng.uniform()
11         S -= np.log(U)
12         X += 1
13     return X
14
15 def sample_erlang(n, lam, rng):
16     """Sum of n Exp(lam) variables for Erl(n, lam)."""
17     U = rng.uniform(0, 1, n)
18     return -np.log(np.prod(U)) / lam    # numerically stable for small n
19
20 # Example
21 samples_poi = [sample_poisson(3.5, rng) for _ in range(10000)]
22 samples_erl = [sample_erlang(4, 1.0, rng) for _ in range(10000)]
23 print(f"Poi(3.5) mean = {np.mean(samples_poi):.3f} (theory: 3.5)")
24 print(f"Erl(4,1) mean = {np.mean(samples_erl):.3f} (theory: 4.0)")
```

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes**
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.4 Uniform Distribution on Subsets of $\mathbb{R}^d$ I

A random vector $X \in A \subset \mathbb{R}^d$ is *uniformly distributed* $\mathcal{U}(A)$ if:

$$\mathbb{P}(X \in D) = \frac{\text{Vol } D}{\text{Vol } A}, \quad D \subset A.$$

Equivalently, $f(x) = \frac{1}{\text{Vol } A} \mathbf{1}(x \in A)$.

Linear transform: If $V \sim \mathcal{U}(a, b)$ is needed, use $V = (b - a)U + a$ where $U \sim \mathcal{U}[0, 1)$.

Proposition 3.4.1 — Rejection for Subsets

If $A \subset B$ and $X \sim \mathcal{U}(B)$, then $(X \mid X \in A) \sim \mathcal{U}(A)$.

3.4 Uniform Distribution on Subsets of $\mathbb{R}^d$ II

Algorithm 13 $\mathcal{U}(A)$:

- 1 Generate i.i.d. $U_1, \dots, U_d \sim \mathcal{U}[0, 1]$
- 2 $V_i \leftarrow (b_i - a_i)U_i + a_i$ (transform to bounding box)
- 3 **if** $V = (V_1, \dots, V_d) \in A$: $X = V$; else go to 1
- 4 **return** X

Curse of Dimensionality for Balls

The d -ball $B_d = \{x : \sum x_i^2 \leq 1\}$ has volume $\text{Vol } B_d = \pi^{d/2} / \Gamma(d/2 + 1)$. The d -cube $A_d = [-1, 1]^d$ has volume 2^d .

Acceptance probability = $\text{Vol } B_d / 2^d$. By Stirling: $\approx \frac{1}{\sqrt{\pi k}} \left(\frac{\pi e}{4k}\right)^k \rightarrow 0$ exponentially fast (e.g., $\approx 2 \times 10^{-14}$ for $d = 30$).

Conclusion: Rejection method is impractical for high-dimensional balls. Better algorithms use the Normal-distribution trick (Section 3.8).

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method**
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.5.1 AR-d (Discrete Acceptance-Rejection) I

Setting: Want $X \sim (p_j)_{j \in E}$. We can easily sample $Y \sim (q_j)_{j \in E}$. There exists $c > 0$ such that

$$p_j \leq c q_j \quad \forall j \in E.$$

Proposition 3.5.1 — AR Correctness

Define the acceptance region $A = \{cUq_Y \leq p_Y\}$ for independent $U \sim \mathcal{U}[0, 1]$ and $Y \sim (q_j)$. Then:

$$\mathbb{P}(A) = 1/c \quad \text{and} \quad \mathbb{P}(Y = j \mid A) = p_j.$$

3.5.1 AR-d (Discrete Acceptance-Rejection) II

Algorithm 14 — AR-d

Require: $c > 0$ with $p_j \leq cq_j$; algorithm to sample $Y \sim (q_j)$.

Ensure: $X \sim (p_j)$.

- 1 **repeat**
 - Generate $Y \sim (q_j)$ and $U \sim \mathcal{U}[0, 1)$ (independent)
- 2 **until** $cUq_Y \leq p_Y$
- 3 **return** $X = Y$

Expected number of iterations = c . Choose the *smallest* possible c .

3.5.1 AR-d (Discrete Acceptance-Rejection) III

Example 3.5.2: Sample X with $p_j = \frac{6}{\pi^2 j^2}$, $j = 1, 2, \dots$

Proposal: $q_j = \frac{1}{j(j+1)}$ (can be sampled as $Y = \lfloor U^{-1} \rfloor$).

$$\frac{p_j}{q_j} = \frac{6}{\pi^2 j^2} \cdot j(j+1) = \frac{6}{\pi^2} \frac{j+1}{j}.$$

$$c = \sup_j (p_j/q_j) = 12/\pi^2.$$

Acceptance condition:

$$cUq_Y \leq p_Y \iff 2UY \leq Y + 1.$$

3.5.2 AR-c (Continuous Acceptance-Rejection) I

Setting: Want X with p.d.f. $f(x)$ on E . Can sample Y with p.d.f. $g(x)$. There exists $c < \infty$ such that $f(x) \leq cg(x)$ for all $x \in E$.

Proposition 3.5.3 — AR-c Correctness

$$\mathbb{P}(A) = 1/c \quad \text{and} \quad \mathbb{P}(Y \in B \mid A) = \int_B f(y) dy.$$

Algorithm 16 — AR-c

Require: $c > 0$ with $f(x) \leq cg(x)$; algorithm to sample $Y \sim g$.

Ensure: $X \sim f$.

- 1 **repeat**
 - Generate $Y \sim g$ and $U \sim \mathcal{U}[0, 1]$ (independent)
- 2 **until** $cUg(Y) \leq f(Y)$
- 3 **return** $X = Y$

3.5.2 AR-c (Continuous Acceptance-Rejection) II

Example 3.5.4 — Normal from Exponential (AR-N)

Half-normal target: $f(x) = \frac{2}{\sqrt{2\pi}} e^{-x^2/2}$, $x > 0$.

Proposal: $g(x) = e^{-x}$ (i.e., $Y \sim \text{Exp}(1)$).

Optimal constant:

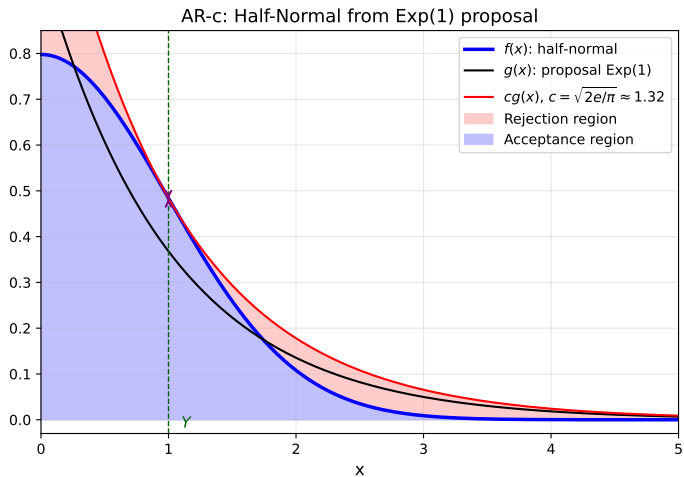
$$c = \max_{x>0} \frac{f(x)}{g(x)} = \sqrt{\frac{2e}{\pi}} \approx 1.32.$$

Acceptance condition (after simplification):

$$U \leq \exp\left(-\frac{(Y-1)^2}{2}\right).$$

Then draw sign $I = \pm 1$ with probability $1/2$ each; return $Z = IY$.

3.5.2 AR-c (Continuous Acceptance-Rejection) III



3.5.2 AR-c (Continuous Acceptance-Rejection) IV

Figure: Half-normal density f , proposal $g \sim \text{Exp}(1)$, and envelope cg . Red area is rejected; blue area is accepted.

3.5 Python Code: AR-c for Normal

Listing 3: AR-c to generate $N(0,1)$ via half-normal and $\text{Exp}(1)$

```
1 import numpy as np
2 rng = np.random.default_rng(42)
3
4 def sample_normal_arc(n, rng):
5     """Algorithm 17 AR-N: generate  $N(0,1)$  via AR-c."""
6     out = np.empty(n)
7     i = 0
8     while i < n:
9         U1 = rng.uniform()
10        Y = -np.log(U1)           #  $Y \sim \text{Exp}(1)$ 
11        U2 = rng.uniform()
12        if U2 <= np.exp(-(Y - 1)**2 / 2): # acceptance condition
13            sign = 1 if rng.uniform() < 0.5 else -1
14            out[i] = sign * Y
15            i += 1
16    return out
17
18 Z = sample_normal_arc(5000, rng)
19 print(f"Mean: {Z.mean():.4f} (theory: 0)")
20 print(f"Std: {Z.std():.4f} (theory: 1)")
```

Unknown normalising constants: When $f(x) = f^*(x)/c_f$ and $g(x) = g^*(x)/c_g$, the acceptance condition $U \leq f^*(Y)/(c^* g^*(Y))$ still works, where $c^* = \sup f^*(x)/g^*(x)$. No need to know c_f or c_g separately.

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method**
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.6 Composition (Mixture) Method I

Setup

Suppose

$$f(x) = \sum_{j=1}^n p_j g_j(x),$$

where g_j are p.d.f.s and (p_j) is a probability function ($p_j \geq 0$, $\sum p_j = 1$). f is then a *mixture distribution*.

Composition Algorithm

- 1 Sample component index J with $\mathbb{P}(J = j) = p_j$.
- 2 Given $J = j$, generate $X \sim g_j$.
- 3 **return** X .

Works for infinite mixtures too: $f(x) = \sum_{j=0}^{\infty} p_j g_j(x)$.

3.6 Composition (Mixture) Method II

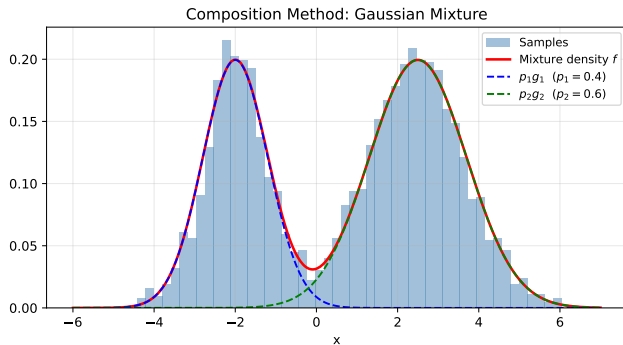


Figure: Gaussian mixture $f = p_1 g_1 + p_2 g_2$ via composition method.

3.6 Composition (Mixture) Method III

Example 3.6.1 — Non-central Chi-square:

$$f(x) = \frac{1}{2} e^{-(x+\lambda)/2} \left(\frac{x}{\lambda}\right)^{d/4-1/2} I_{d/2-1}(\sqrt{\lambda x}), \quad x \geq 0.$$

Can be represented as a mixture of $\text{Gamma}(j + d/2, 1/2)$ distributions with $\text{Poisson}(\lambda/2)$ mixing weights.

Example 3.6.3 — Gamma-Poisson (Negative Binomial) mixture:

If $X \mid \Lambda \sim \text{Poi}(\Lambda)$ and $\Lambda \sim \text{Gamma}(\alpha, \lambda)$, then

$$X \sim \text{NB}\left(\alpha, \frac{1}{1+\lambda}\right).$$

This is the *Gamma-Poisson mixture* representation.

3.6 Python Code: Mixture Sampling

Listing 4: Composition method for Gaussian mixture

```
1  import numpy as np
2  rng = np.random.default_rng(0)
3
4  def sample_mixture(p, mus, sigmas, n, rng):
5      """
6          Composition method for Gaussian mixture.
7          p      : list of mixture weights (sum to 1)
8          mus    : list of means
9          sigmas : list of std devs
10         """
11         p = np.array(p)
12         # Step 1: sample component index
13         J = rng.choice(len(p), size=n, p=p)
14         # Step 2: sample from chosen component
15         X = np.array([rng.normal(mus[j], sigmas[j]) for j in J])
16         return X
17
18     p      = [0.3, 0.5, 0.2]
19     mus    = [-3.0, 1.0, 5.0]
20     sigmas = [0.8, 1.2, 0.5]
21     X = sample_mixture(p, mus, sigmas, n=10000, rng=rng)
22     print(f"Empirical mean: {X.mean():.3f}")
23     # Theory: 0.3*(-3) + 0.5*1 + 0.2*5 = 0.6
```

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers**
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.7.1 Box-Muller Method I

Lemma 3.7.1 — Polar Representation of Bivariate Normal

Let Z_1, Z_2 be i.i.d. $\mathcal{N}(0, 1)$. Let (D, Θ) be their polar coordinates. Then:

- D and Θ are *independent*,
- D has p.d.f. $f(r) = re^{-r^2/2}$ (Rayleigh distribution),
- $\Theta \sim \mathcal{U}[0, 2\pi)$.

Lemma 3.7.2 — Rayleigh Distribution

If D has the Rayleigh distribution, then $Y = D^2 \sim \text{Exp}(1/2)$. Thus $D \stackrel{\mathcal{D}}{=} (-2 \log U)^{1/2}$.

3.7.1 Box-Muller Method II

Proposition 3.7.3 — Box-Muller

Let $U_1, U_2 \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}[0, 1)$. Set

$$Z_1 = (-2 \log U_1)^{1/2} \cos(2\pi U_2), \quad Z_2 = (-2 \log U_1)^{1/2} \sin(2\pi U_2).$$

Then Z_1, Z_2 are i.i.d. $\mathcal{N}(0, 1)$.

3.7.1 Box-Muller Method III

Algorithm 20 (BM):

- 1 Generate U_1, U_2 i.i.d. $\mathcal{U}[0, 1)$
- 2 $D \leftarrow -2 \log U_1, V \leftarrow 2\pi U_2$
- 3 $Z_1 = \sqrt{D} \cos V, Z_2 = \sqrt{D} \sin V$
- 4 **return** Z_1, Z_2

3.7.1 Box-Muller Method IV

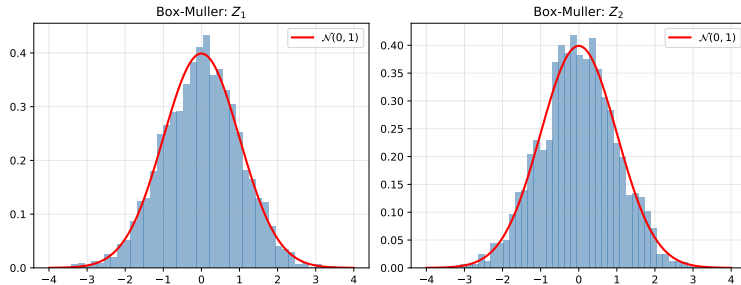


Figure: Box-Muller: histograms of Z_1 and Z_2 vs $\mathcal{N}(0, 1)$ density.

3.7.1 Marsaglia-Bray Method I

Proposition 3.7.4 — Marsaglia-Bray

Let $V_1, V_2 \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}[-1, 1)$. Set $Y_i = (-2 \log(V_1^2 + V_2^2))^{1/2} V_i / (V_1^2 + V_2^2)^{1/2}$.

Conditional on $V_1^2 + V_2^2 \leq 1$: $(Z_1, Z_2) = (Y_1, Y_2) \stackrel{D}{=} \text{i.i.d. } \mathcal{N}(0, 1)$.

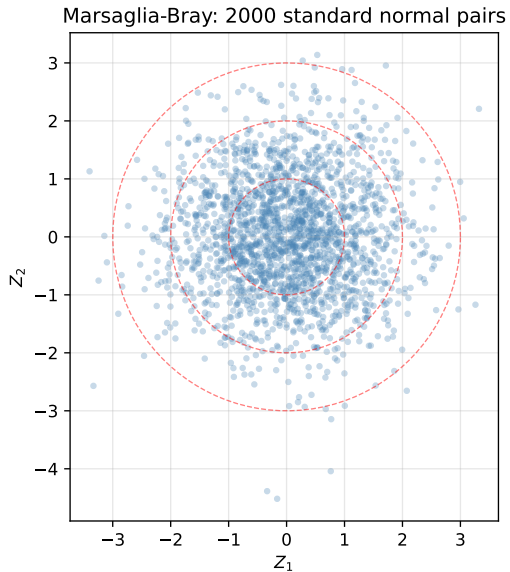
Algorithm 21 (MB):

- 1 **repeat:** U_1, U_2 i.i.d. $\mathcal{U}[0, 1)$; $V_1 = 2U_1 - 1$, $V_2 = 2U_2 - 1$; $X' = V_1^2 + V_2^2$
- 2 **until** $X' \leq 1$
- 3 $Y \leftarrow \sqrt{-2 \log(X')/X'}$
- 4 $Z_1 = V_1 \cdot Y$, $Z_2 = V_2 \cdot Y$

Acceptance probability $\approx \pi/4 \approx 78.5\%$.

Advantage over Box-Muller: Avoids trigonometric functions; generally faster in practice.

3.7.1 Marsaglia-Bray Method II



3.7.2 ITM for Normal Variables

To generate $\mathcal{N}(0,1)$ via ITM, invert the c.d.f. Φ . No closed form for $\Phi^{-1}(u)$; use numerical methods.

Newton's Method

Solve $\Phi(x) = u$ starting from $x_0 = \pm\sqrt{|\log(1.0004 - (1 - 2u)^2)|}$:

$$x_{n+1} = x_n + (u - \Phi(x_n))e^{0.5x_n^2 + c}, \quad c = \log \sqrt{2\pi}.$$

One additional step reduces error to below 10^{-15} .

Listing 5: Inverse normal c.d.f. via scipy

```
1 from scipy.stats import norm
2 import numpy as np
3
4 # scipy.stats.norm.ppf implements the Beasley-Springer-Moro algorithm
5 p = 0.95
6 quantile = norm.ppf(p)
7 print(f"Phi^(-1)(0.95) = {quantile:.6f}") # 1.644854
8
9 # Generate N(0,1) samples via ITM
10 rng = np.random.default_rng(42)
11 U = rng.uniform(0, 1, 5000)
12 Z = norm.ppf(U) # inverse CDF method
```

3.7.3 Generating Multivariate Normal $\mathcal{N}(\mu, \Sigma)$ I

Cholesky Factorisation Method

Every symmetric positive-semidefinite matrix Σ admits

$$\Sigma = AA^T.$$

(Cholesky gives lower-triangular A .)

Proposition 3.7.8: Let $Z = (Z_1, \dots, Z_d)^T$, i.i.d. $\mathcal{N}(0, 1)$. Then

$$X = AZ + \mu \sim \mathcal{N}(\mu, \Sigma).$$

Proof: $\mathbb{E}[X] = \mu$ and $\text{Cov}(X) = A \text{Id } A^T = \Sigma$.

3.7.3 Generating Multivariate Normal $\mathcal{N}(\mu, \Sigma)$ II

Bivariate Case ($d = 2$)

With $\Sigma = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}$:

$$X_1 = \sigma_1(\sqrt{1 - |\rho|}Z_1 + \sqrt{|\rho|}Z_3), \quad X_2 = \sigma_2(\sqrt{1 - |\rho|}Z_2 + \text{sign}(\rho)\sqrt{|\rho|}Z_3).$$

3.7.3 Generating Multivariate Normal $\mathcal{N}(\mu, \Sigma)$ III

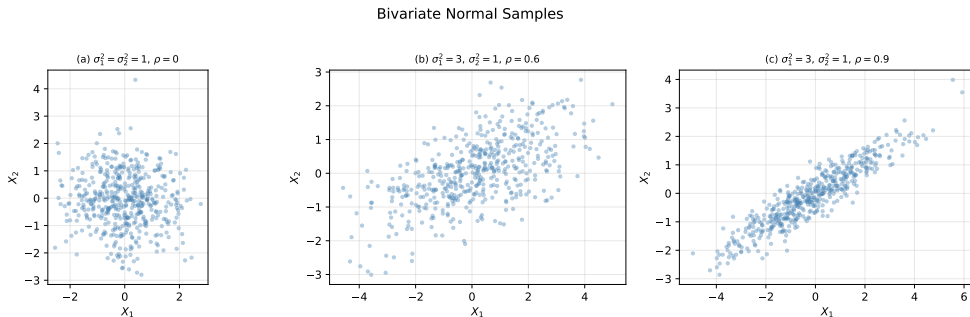


Figure: 500 samples from bivariate normal under three different covariance settings.

3.7.3 Python: Multivariate Normal via Cholesky

Listing 6: Generate multivariate normal via Cholesky decomposition

```
1 import numpy as np
2 rng = np.random.default_rng(42)
3
4 def sample_multivariate_normal(mu, Sigma, n, rng):
5     """Generate n samples from N(mu, Sigma) via Cholesky."""
6     d = len(mu)
7     A = np.linalg.cholesky(Sigma)    # Sigma = A @ A.T
8     Z = rng.standard_normal((d, n))  # d x n matrix of i.i.d. N(0,1)
9     return (A @ Z).T + mu            # n x d output
10
11 # Example: 2D normal with correlation 0.8
12 mu = np.array([1.0, -2.0])
13 Sigma = np.array([[2.0, 1.2],
14                   [1.2, 1.0]])      # rho = 1.2/sqrt(2) ? 0.849
15 samples = sample_multivariate_normal(mu, Sigma, 2000, rng)
16 print("Empirical mean:", samples.mean(axis=0).round(3))
17 print("Empirical cov:\n", np.cov(samples.T).round(3))
```

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.8 Geometric Setup I

Objects:

$$B_d = \{x \in \mathbb{R}^d : x_1^2 + \cdots + x_d^2 \leq 1\}, \quad \text{Vol}(B_d) = \frac{\pi^{d/2}}{\Gamma(d/2 + 1)},$$

$$S_{d-1} = \{x \in \mathbb{R}^d : x_1^2 + \cdots + x_d^2 = 1\}, \quad \text{Area}(S_{d-1}) = \frac{d\pi^{d/2}}{\Gamma(d/2 + 1)}.$$

Change of Variables — Key Formula

For a parametrised surface $x(t)$ with Jacobian matrix $J(t) = (\partial x_j / \partial t_i)_{i,j}$:

$$\sqrt{g(t)} = \sqrt{|\det(JJ^T)|}.$$

Surface area element: $dA = \sqrt{g(t)} dt$.

3.8 Geometric Setup II

3.8.1 Random Point in B_3 and on S_2

Spherical coordinates for B_3 : $x = r \cos \theta \sin \phi$, $y = r \sin \theta \sin \phi$, $z = r \cos \phi$.

Proposition 3.8.4: For $X \sim \mathcal{U}(B_3)$ in spherical coords (D, Φ, Θ) :

- $D^{1/3} \sim \mathcal{U}[0, 1)$ (so $D \stackrel{\mathcal{D}}{=} U^{1/3}$),
- $\Phi = \arccos(1 - 2U')$, $U' \sim \mathcal{U}[0, 1)$,
- $\Theta \sim \mathcal{U}[0, 2\pi)$.

Proposition 3.8.6: If $Z_1, Z_2, Z_3 \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$:

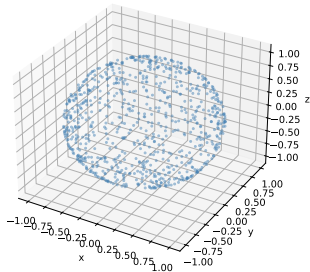
$$\left(\frac{Z_1}{\|Z\|}, \frac{Z_2}{\|Z\|}, \frac{Z_3}{\|Z\|} \right) \sim \mathcal{U}(S_2).$$

Proposition 3.8.13: If $X \sim \mathcal{U}(S_{d-1})$ and $U \sim \mathcal{U}[0, 1)$ independent, then $U^{1/d}X \sim \mathcal{U}(B_d)$.

3.8 Geometric Setup III

Random Points on Sphere and in Ball

Uniform on S_2 (sphere)



Uniform in B_3 (ball)

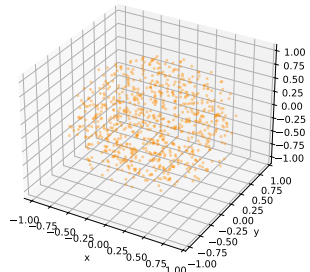


Figure: Uniform random points on S_2 (left) and in B_3 (right).

3.8 Python: Sphere and Ball Sampling

Listing 7: Sample uniformly from sphere and ball in d dimensions

```
1 import numpy as np
2 rng = np.random.default_rng(42)
3
4 def sample_sphere(d, n, rng):
5     """Uniform on (d-1)-sphere  $S_{\{d-1\}}$  by normalising  $N(0,I)$ ."""
6     Z = rng.standard_normal((n, d))
7     return Z / np.linalg.norm(Z, axis=1, keepdims=True)
8
9 def sample_ball(d, n, rng):
10    """Uniform in d-ball  $B_d$ :  $X = U^{\{1/d\}} * \text{point\_on\_sphere}$ ."""
11    X = sample_sphere(d, n, rng)
12    U = rng.uniform(0, 1, n)
13    return X * U[:, None]**(1.0/d)
14
15 # 3D examples
16 pts_sphere = sample_sphere(3, 1000, rng)
17 pts_ball = sample_ball(3, 1000, rng)
18 print("Sphere norms:", np.linalg.norm(pts_sphere, axis=1).mean().round(4)) # ~1
19 print("Ball norms: ", np.linalg.norm(pts_ball, axis=1).mean().round(4)) # ~0.75
```

3.8.2–3.8.3 Spherical Coordinates & Ellipsoids I

3.8.2 General Spherical Coordinates in $\mathbb{R}^d$

$$x_1 = r \cos \phi_1, \quad x_2 = r \sin \phi_1 \cos \phi_2, \quad \dots$$

Jacobian: $J = r^{d-1} \sin^{d-2} \phi_1 \sin^{d-3} \phi_2 \cdots \sin \phi_{d-2}$.

Lemma 3.8.10: For $Z \sim \mathcal{N}(0, \text{Id})$, the spherical coordinates $(D, \Phi_1, \dots, \Phi_{d-2}, \Theta)$ are independent, $D^2 \sim \chi_d^2$, and Φ_i has p.d.f. $c_{d-i-1} \sin^{d-i-1} x$.

3.8.2–3.8.3 Spherical Coordinates & Ellipsoids II

3.8.3 Ellipsoids

A d -ellipsoid: $\mathcal{E}_d = \{x \in \mathbb{R}^d : x^T A^{-1} x \leq 1\}$ where A is symmetric positive-definite.

Factorize $A = LL^T$.

Algorithm 23:

- 1 Diagonalize $A = P \operatorname{diag}(a^2) P^T$
- 2 $L \leftarrow P \operatorname{diag}(a)$
- 3 Generate $X \sim \mathcal{U}(B_d)$
- 4 $Y \leftarrow LX$
- 5 **return** Y

3.8.4–3.8.5 Sampling on an Ellipsoid Surface I

Problem: Sample uniformly from the surface of the ellipsoid $\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} = 1$.

Parametrisation: $\mathbf{x}(\theta, \phi) = (a \cos \theta \sin \phi, b \sin \theta \sin \phi, c \cos \phi)$ with $\theta \in [0, 2\pi)$, $\phi \in [0, \pi)$.

$$\sqrt{g(\phi, \theta)} = \sin \phi \sqrt{c^2(a^2 - b^2) \cos^2 \theta (\cos^2 \phi - 1) + a^2(b^2 - c^2) \cos^2 \phi + c^2}.$$

Density $\propto \sqrt{g(\phi, \theta)}$ — coordinates are *not* independent for general a, b, c .

Numerical Approach (Algorithm 24)

- 1 Build a grid of $n_k \times n_l$ parallelogram patches on the surface.
- 2 Approximate each patch area using cross-products of tangent vectors.
- 3 Sample a patch proportional to its area (composition method).
- 4 Sample a uniform point within the chosen patch.

3.8.4–3.8.5 Sampling on an Ellipsoid Surface II

3.8.5 Average Shadow of a Convex 3D Body

For a flat face F with outward normal randomly drawn from $\mathcal{U}(S_2)$:

$$\mathbb{E}[\text{Area}(\text{shadow}(F))] = \frac{\text{Area}(F)}{2}.$$

For a convex polyhedron B with faces $F_1, \dots, F_m$:

$$\mathbb{E}[\text{Area}(\text{shadow}(B))] = \frac{\text{Area}(\text{surface}(B))}{4}.$$

For a cube with edge s : $\mathbb{E}[\text{shadow}] = \frac{6s^2}{4} = \frac{3}{2}s^2$.

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

3.9.1 Conditional Inverse Transform Method I

Setup: Joint density $f(x, y)$, marginals $f_X(x)$ and $f_Y(y)$, conditional c.d.f.:

$$F_{Y|X=x}(y) = \frac{F_{X=x,Y}(x, y)}{f_X(x)}, \quad F_{X=x,Y}(x, y) = \int_{y_0}^y f(x, u) du.$$

Algorithm: Conditional ITM

- 1 Compute marginal c.d.f. F_X ; sample $X = F_X^{-1}(U_1)$.
- 2 Given $X = x$, compute conditional c.d.f. $F_{Y|X=x}$.
- 3 Set $Y = F_{Y|X=x}^{-1}(U_2)$, independent U_2 .
- 4 **return** (X, Y) .

3.9.1 Conditional Inverse Transform Method II

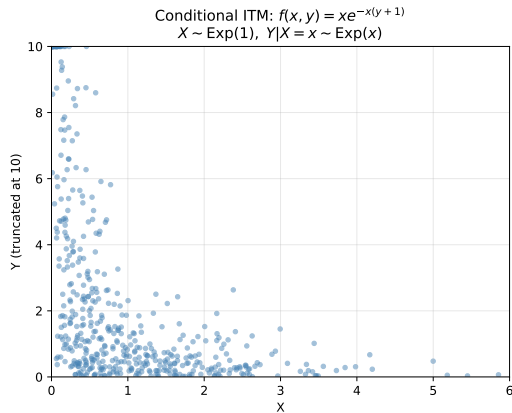


Figure: 100 samples from $f(x, y) = xe^{-x(y+1)}$ via conditional ITM.

3.9.1 Conditional Inverse Transform Method III

Example 3.9.1: $f(x, y) = x \exp(-x(y + 1))$, $x, y \geq 0$.

- $f_X(x) = e^{-x}$, so $X \sim \text{Exp}(1)$: $X = -\log U_1$.
- $f_{Y|X=x}(y) = xe^{-xy}$ ($\text{Exp}(x)$): $Y = -\log U_2/X$.

3.9.2 Bivariate Copulas — Sklar's Theorem I

Definition: Copula

A **copula** $C(u_1, u_2)$ is any bivariate c.d.f. with $\mathcal{U}[0, 1)$ marginals. Formally, $C : [0, 1]^2 \rightarrow [0, 1]$ satisfies:

- 1 $C(u_i, 0) = 0$ and $C(u_i, 1) = u_i$,
- 2 $C(v_1, v_2) - C(u_1, v_2) - C(v_1, u_2) + C(u_1, u_2) \geq 0$ for $u_i \leq v_i$.

3.9.2 Bivariate Copulas — Sklar's Theorem II

Theorem 3.9.2 — Sklar's Theorem

For any joint c.d.f. $F_{X,Y}(t_1, t_2)$ there exists a copula C such that

$$F_{X,Y}(t_1, t_2) = C(F_X(t_1), F_Y(t_2)).$$

If the marginals F_X and F_Y are continuous, C is *unique*.

Moreover, the joint density factors as:

$$f_{X,Y}(t_1, t_2) = f_X(t_1) f_Y(t_2) c(F_X(t_1), F_Y(t_2)),$$

where c is the copula density.

3.9.2 Bivariate Copulas — Sklar's Theorem III

Frchet-Hoeffding Bounds (Theorem 3.9.6)

For any copula C :

$$(F(x) + F(y) - 1)_+ \leq F(x, y) \leq \min(F(x), F(y)).$$

- **Upper bound** $C_U(u_1, u_2) = \min(u_1, u_2)$: perfect positive dependence, $V_1 = V_2 = U$.
- **Lower bound** $C_L(u_1, u_2) = \max(0, u_1 + u_2 - 1)$: perfect negative dependence, $V_1 = U$, $V_2 = 1 - U$.
- **Independence copula** $C_I(u_1, u_2) = u_1 u_2$: $V_1 = U_1$, $V_2 = U_2$ independent.

3.9.2 Bivariate Copulas — Sklar's Theorem IV

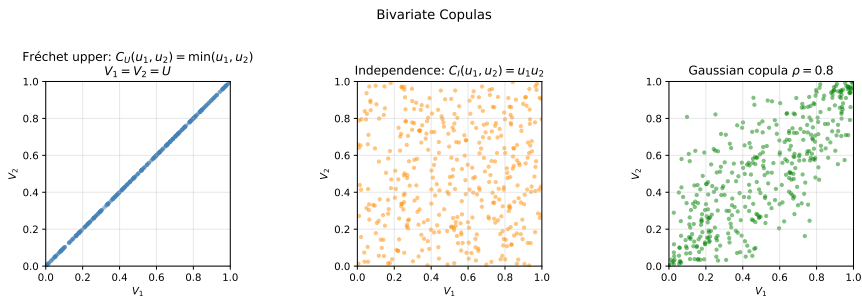


Figure: Fréchet upper, independence, and Gaussian ($\rho = 0.8$) copula samples.

3.9.3 Classical Copulas I

Gaussian (Normal) Copula

Let Φ_ρ be the bivariate normal c.d.f. with correlation ρ :

$$C(u_1, u_2) = \Phi_\rho(\Phi^{-1}(u_1), \Phi^{-1}(u_2)).$$

Simulation: Generate $(X, Y) \sim \mathcal{N}(0, \Sigma)$ with $\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$; set $(V_1, V_2) = (\Phi(X), \Phi(Y))$.

- As $\rho \rightarrow 0$: $C \rightarrow C_I$; as $\rho \rightarrow 1$: $C \rightarrow C_U$; as $\rho \rightarrow -1$: $C \rightarrow C_L$.

Clayton Copula (Exercise 3.T.38)

Parameter $\theta > 0$: $(V_1, V_2) = ((1 - \log U_1/X)^{-1/\theta}, (1 - \log U_2/X)^{-1/\theta})$, where $X \sim \text{Gamma}(1/\theta, 1)$ and $U_1, U_2 \sim \mathcal{U}[0, 1]$ independent.

3.9.3 Classical Copulas II

Gaussian Copula Samples

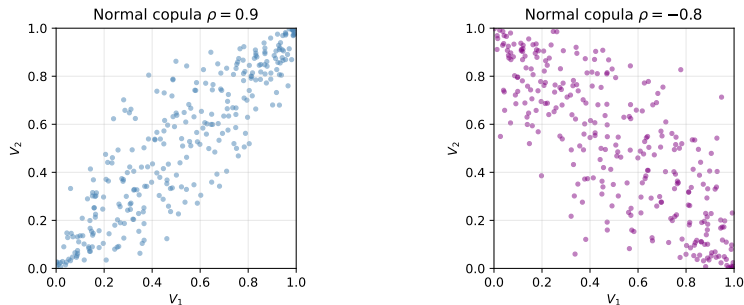


Figure: Gaussian copula: $\rho = 0.9$ (left) and $\rho = -0.8$ (right).

Using Copulas to Construct Joint Distributions (Example 3.9.10):

Given $\text{Exp}(1)$ marginal for X' and $\text{Par}(1)$ for Y' :

3.9.3 Classical Copulas III

- 1 Sample (V_1, V_2) from a Gaussian copula with ρ .
- 2 Transform: $X' = F_{\text{Exp}}^{-1}(V_1) = -\log(1 - V_1)$, $Y' = F_{\text{Par}}^{-1}(V_2) = \frac{1}{1-V_2} - 1$.

This preserves the marginal distributions while changing the dependence structure.

3.9 Python: Gaussian Copula

Listing 8: Gaussian copula with specified marginals

```
1 import numpy as np
2 from scipy.stats import norm
3
4 rng = np.random.default_rng(42)
5
6 def sample_gaussian_copula(rho, n, rng):
7     """Sample (V1, V2) from Gaussian copula with correlation rho."""
8     Sigma = np.array([[1, rho], [rho, 1]])
9     A = np.linalg.cholesky(Sigma)
10    Z = rng.standard_normal((2, n))
11    W = A @ Z
12    return norm.cdf(W[0]), norm.cdf(W[1])
13
14 def to_exp_pareto(V1, V2):
15     """Transform copula samples to Exp(1) x Pareto(1) marginals."""
16     X = -np.log(1 - V1) # F_Exp^{-1}(V1)
17     Y = 1.0 / (1 - V2) - 1 # F_Par^{-1}(V2) for Par(alpha=1)
18     return X, Y
19
20 V1, V2 = sample_gaussian_copula(rho=0.8, n=1000, rng=rng)
21 X, Y = to_exp_pareto(V1, V2)
22 print(f"Corr(X,Y) = {np.corrcoef(X, Y)[0,1]:.3f}")
```

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

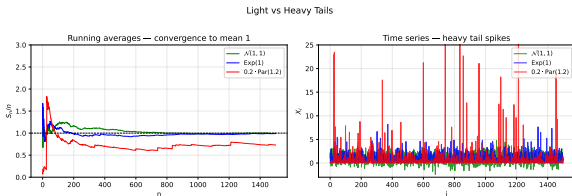
3.10 Can We See the Heaviness of a Distribution? I

Setting: Compare i.i.d. sequences $X_1, X_2, \dots$ with the same mean ($= 1$) but different tails. Look at:

- Running average $S_n/n = \frac{1}{n} \sum_{i=1}^n X_i$,
- Individual time series (X_i).

Light vs Heavy Tails

Distribution	Mean	Tail
$\mathcal{N}(1, 1)$	1	Light
$\text{Exp}(1)$	1	Light
$\text{LN}(-1/2, 1)$	1	Heavy
$\text{W}(1/2, \sqrt{2})$	1	Heavy
$0.2 \cdot \text{Par}(1.2)$	1	Very heavy



3.10 Can We See the Heaviness of a Distribution? II

QQ Plots for Tail Detection

Plot the sample quantiles of X against theoretical quantiles of $\mathcal{N}(0, 1)$:

- **Light-tailed:** points approximately on a straight line.
- **Heavy-tailed:** upper right (and lower left) bend upward.
- **Skewed:** asymmetric deviations.

3.10 Can We See the Heaviness of a Distribution? III

QQ Plots: Detecting Tail Behaviour

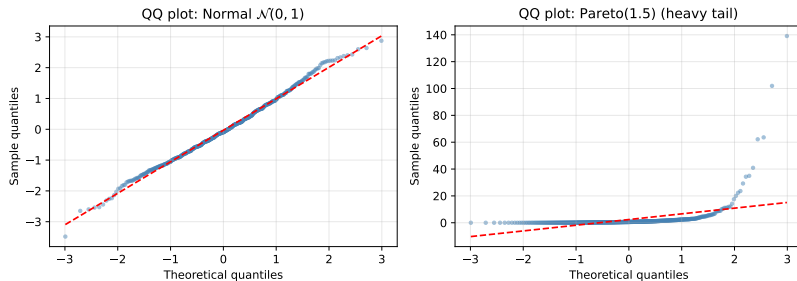


Figure: QQ plots: Normal (left, straight line) vs Pareto(1.5) (right, heavy-tail bend).

Key observation: The Pareto distribution has *infinite variance* for $\alpha \leq 2$; this makes S_n/n converge extremely slowly, with large spikes.

Outline

- 1 3.1 Inverse Transform Method (ITM)
- 2 3.2 ITM for Lattice Distributions
- 3 3.3 Ad Hoc Methods
- 4 3.4 Uniform Distribution on Geometric Shapes
- 5 3.5 Acceptance-Rejection Method
- 6 3.6 Composition Method
- 7 3.7 Generating Gaussian Random Numbers
- 8 3.8 Uniform Distribution on Balls, Spheres, Ellipsoids

Chapter 3 — Summary of Methods I

Chapter 3 — Summary of Methods II

Method Overview

Method	When to use	Key idea
ITM (Alg. 6)	$F^{\leftarrow}$ available	$X = F^{\leftarrow}(U)$
ITM-d (Alg. 9)	Discrete, finite mean	Sequential accumulation
ITR (Alg. 10)	$(a, b, 0)$ class PMFs	Ratio recursion
Geo. closed form	$\text{Geo}(p)$	$X = \lfloor \log U / \log p \rfloor$
DB (Alg. 11)	Binomial $B(n, p)$	Sum of Bernoullis
Erlang	$\text{Erl}(n, \lambda)$	Product of Uniforms
DP (Alg. 12)	Poisson $\text{Poi}(\lambda)$	Renewal process
AR-d (Alg. 14)	Discrete, $p_j \leq cq_j$	Accept with prob $p_j/(cq_j)$
AR-c (Alg. 16)	Continuous, $f \leq cg$	Geometric area under curve
Composition	Mixture $f = \sum p_j g_j$	Sample component, then value
Box-Muller (Alg. 20)	Normal pairs	Polar transform
Marsaglia-Bray (Alg. 21)	Normal pairs	Polar in 2D ball
Cholesky	Multivariate normal	$X = AZ + \mu$
Normal trick	$\mathcal{U}(S_{d-1})$	Normalize $\mathcal{N}(0, \text{Id})$
Cond. ITM	Joint densities	Marginal then conditional
Cond. ITR	Flexible dependencies	Slide the screen

3.11 Bibliographical Notes

- **Devroye (1986)** — *Non-Uniform Random Variate Generation*. Comprehensive reference for all methods in this chapter.
- **Knuth (1997)** — *The Art of Computer Programming, Vol. 2*. Classic treatment of the inverse transform method.
- **Asmussen & Glynn (2007)** — Fréchet-Hoeffding bounds and copulas.
- **Box & Muller (1958)** — Original Box-Muller algorithm paper.
- **Marsaglia & Bray (1964)** — Polar method (Marsaglia-Bray).
- **Glasserman (2004)** — ITM for normal r.v.s and Beasley-Springer-Moro.
- **Madras (2002)** — AR-c for Gamma distribution.
- **Wichura (1988), AS241** — Algorithm behind `scipy.stats.norm.ppf`.
- **Embrechts & Hofert** — Properties of copulas and generalized inverses.
- **Dagpunar (2007)** — Composition method, Gamma sampling.